

BRT Safety Filtering for LLM-Generated Orbital Maneuver Plans

Tanis Priddle

AA276, Spring 2026

Problem:

There is increasing interest in the use of LLMs in autonomous mission planning and in the generation of maneuver sequences from natural language objectives; however, LLMs have no internal model of orbital mechanics or physical safety constraints (No formal framework exists that intercepts and corrects unsafe maneuver commands before execution!)



Motivation

- Autonomous on-board planning is becoming a higher priority for deep-space and congested-LEO missions where ground-in-the-loop latency is restrictive
- LLMs can articulate flexible mission intent in natural language but don't provide any safety guarantees -> we need a safety layer before actuation!
- Closing the gap between semantic planning (what the mission wants) and formal safety (what physics allows)

Outcome

A pipeline where an LLM generates proximity maneuver plans from natural language mission specs, and a precomputed Backward Reachable Tube (BRT) acts as a safety filter to replace unsafe commanded states with minimum-norm safe alternatives.

Approach

- Model relative motion between chaser and target using Clohessy-Wiltshire (CW) equations: 6D state

- Precompute the BRT offline using the h_j -reachability grid-based solver, define the failure set as entry into an ellipsoidal keep-out zone around the target

- Prompt an LLM (via API) with mission specs to produce maneuver plans as JSON: {timestep, ΔV vector}.

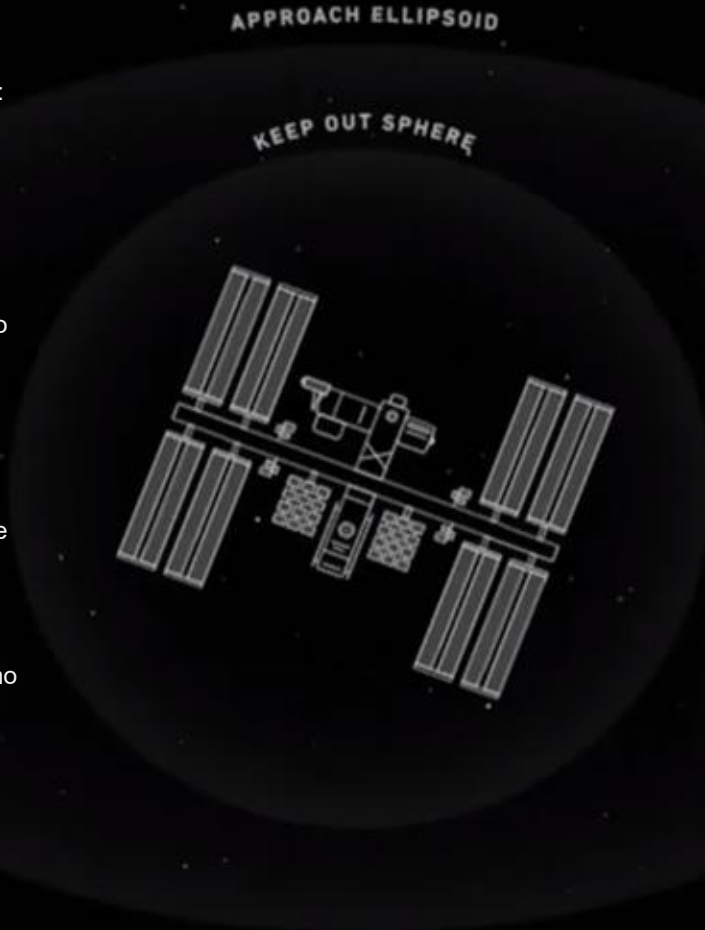
 - Vary prompts across approach angle, urgency, range to failure modes, etc

- Evaluate $V(x)$ from the BRT at each step: if the resulting state is inside the BRT (unsafe), replace the command with the minimum-norm projection onto the BRT boundary

- Baseline comparison: same set of plans with no filter applied, and a simple rule-based planner (fixed radial burn) as a non-LLM anchor

- Metrics: fraction of plans intercepted by the filter, ΔV overhead introduced by corrections, mission success rate (did chaser reach target?) before and after filtering

 - Evaluate on 50-100 LLM-generated plans across varied scenarios.



Milestones

Week 6: Set up CW dynamics, define ellipsoidal KOZ

Week 7: Compute BRT using h_j -reachability, verify BRT boundary, implement filter

Week 8: Build LLM prompting pipeline, run filter over all plans

Week 9: Evaluate metrics, run baseline comparisons, test edge cases (short horizon, ambiguous prompts)

Week 10: Debugging and final testing, final report and presentation!